`node`; it returns the longest prefix of `s` stored in the trie.

#2 Check if the string searched is empty: if it is, since this method is implemented using recursion, we know it has traversed the whole path in the trie: this is the node exactly matching `s`.

#3 Since `s` is not the empty string, we can see if there is an edge matching it, even partially.

#4 Match case #4 (WRT figure 6.15): the searched prefix is not stored in the tree.

#5 Match case #1. There is an edge whose label is a prefix to `s`: we just need to traverse the edge and recurse on the remaining characters in the string.

#6 Match case #3: although there is no node storing at the end of the path for the searched prefix, there would be a pass-through node for it in the uncompressed trie. This means that all prefixes stored will start with `s+edgeSuffix`, and those will be the only strings stored having `s` as a prefix.

#7 Match case #4: there is no path starting with `s` in the (sub)trie rooted at `node`.

This concludes our rundown of the main methods for radix tries, and the discussion on data structures for efficient strings search; to the interested reader that would like to delve further into this subject, we suggest taking a look at suffix trees and suffix arrays, two interesting data structures, fundamental for fields like bioinformatics, that, unfortunately, are out of scope for this chapter.

6.4 Applications